# GUÍA COMPLETA DE HERENCIA Y COMPOSICIÓN EN OOP

## 4. HERENCIA

**Definición:** mecanismo por el cual una clase (**subclase**) reutiliza y extiende los atributos y métodos de otra (**superclase**). Modela la relación "**es un**".

**SUPERCLASE** (base)

**protected** String nombre;
**protected double** salarioBase;

```
public abstract class Empleado {
    protected String nombre;
    protected double salarioBase;
    public abstract double calcularPago();
}
```

**SUBCLASE** (derivada)

**private double** bono;
**public double** calcularPago() { .... }

```
public class Gerente extends Empleado {
    private double bono;
    @Override
    public double calcularPago() { ... }
}
```

### DETALLES DE JAVA & super



Java permite **una sola** superclase, pero muchas interfaces.

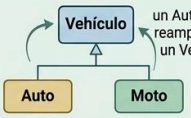
**superclase:**

**super** invoca al constructor o método de la superclase.

```
super.constructor()
```

### REGLAS PRÁCTICAS & LISKOV

Usa **herencia** solo si se cumple **"X es un Y"** y la subclase puede sustituir a la superclase sin sorpresas (Principio de Sustitución de Liskov).




un Auto puede reemplazarrar un Vehículo.

### HERENCIA vs. COMPOSICIÓN ("tiene un")

**COMPOSICIÓN ("tiene un"):**

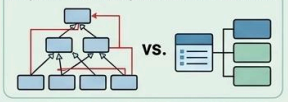
```
class Auto { private Motor motor; }
```




Auto **TIENE** un Motor, no **ES** un Motor

**REGLA DE ORO DE ARQUITECTURA favorece la composición sobre la herencia.**

Las jerarquías profundas generan acoplamiento fuerte y son difíciles de cambiar.




**Práctica (15 min):**
Diseña la jerarquía Vehiculo → Auto, Moto, Camion. Luego discute: ¿Camion que transporta Carga debe heredar de Carga? ¿Por qué no?

# 3. EL ENCAPSULAMIENTO EN LA PROGRAMACIÓN ORIENTADA A OBJETOS

## DEFINICIÓN Y EJEMPLO CÓDIGO

### CONCEPTO Y CÓDIGO

**Definición:** agrupar datos y comportamiento en una unidad y no y **controlar el acceso** al estado interno, exponiendo solo lo necesario.



```
public class Cuenta {
    private double saldo;          ← DATOS PROTEGIDOS

    public void retirar(double monto) {
        if (monto <= 0 || monto > saldo)
            throw new IllegalArgumentException("Monto inválido");
        saldo -= monto;
    }
    public double getSaldo() { return saldo; }   ← COMPORTAMIENTO EXPUESTO
}
```

COMPORTAMIENTO EXPUESTO

## TABLA DE MODIFICADORES DE ACCESO

### MODIFICADORES DE ACCESO (VISIBILIDAD)

| Modificador | Clase | Paquete | Subclase | Todos |
|---|---|---|---|---|
| private | ✓ | | | ✓ |
| (default) | ✓ | ✓ | | |
| protected | ✓ | ✓ | ✓ | |
| public | ✓ | ✓ | ✓ | ✓ |

Diferencia niveles de acceso

CONCEPTUAL LEGENDA:
- ✓ Protege caldor (ej. saldo nunca negativo)
- ✓ Permite cambiar implementación sin afectar clientes
- ✓ Permite cambiar implementación sin oir afectar visibilidad)
- ✓ Facilita las pruebas (ao puecas (Testabilidad)

## BENEFICIOS Y ERRORES COMUNES

### BENEFICIOS Y BUENAS PRÁCTICAS

- ***Beneficios:***
  - Protege invariantes (ej. saldo nunca negativo)
  - Permite cambiar implementación sin afectar clientes
  - Facilita las pruebas (Testabilidad)
- ***Error común:*** Generar getters y setters para todo. Eso es "exposición disfrazada".

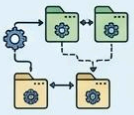
  Expón comportamientos (como 'retirar'), no datos crudos (como 'setSaldo').

## EN ARQUITECTURA Y PRÁCTICA

### ENCAPSULAMIENTO EN LA PRÁCTICA Y ARQUITECTURA




- ***En arquitectura:*** Cada módulo o capa encapsula sus detalles; solo publica una API estable.



- ***Práctica*** **(15 min):** Corrige una clase Producto con atributos públicos: hazlos privados, valida que el precio sea negativo y agrega un método **aplicarDescuento**(porcentaje).




```
Producto {
    private saldo;
    private = saldo;
}
void Producto {
    private setSoot();
    precio = precivo;
}
```

# 2. LA ABSTRACCIÓN EN LA PROGRAMACIÓN ORIENTADA A OBJETOS Y ARQUITECTURA DE SOFTWARE

## DEFINICIÓN: Enfocarse en lo ESENCIAL



- Lista, hobbies,
- Hobbes, Mascota,
- Caro,
- Color favorito,
- Pasatiempo,
- Paaziomco, etc.

### MODELO "EMPLEADO" para NÓMINA



Nombre | Datos
Salario | Cargo
Cargo | ...

DATOS ESENCIALES (RELEVANTES)

### IRRELEVANTE




## HERRAMIENTAS EN JAVA: CONCRETANDO LA ABSTRACCIÓN

CLASES ABSTRACTAS | INTERFACES

## EN ARQUITECTURA: Interfaces como CONTRATOS



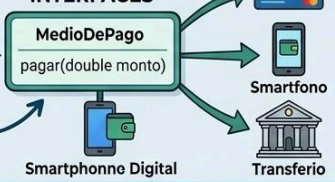

"programar hacia interfaces, no hacia implementaciones"

Capa de Interfaz




Quien usa MedioDePago la saber la implementación

### INTERFACES




## PRACTICA (15 min): ¡HAZLO TÚ MISMO!

1. ELIGE UN SISTEMA




2. IDENTIFICA 3 CLASES

3. LISTA 3 ATRIBUTOS & MÉTODOS ESENCIALES

4. DESCARTA LO IRRELEVANTE






# LOS CUATRO PILARES EN LA ARQUITECTURA DE SOFTWARE

| Pilar | Problema que resuelva | Efecto en la arquitectura |
|---|---|---|
| Abstracción | Complejidad | Contratos e interfaces entre capas |
| Encapsulamiento | Dependencia de detalles internos | Módulos con bajo acoplamiento |
| Herencia | Duplicación de código | Reutilización (con cuidado) |
| Polimorfismo | Código rígido | Extensibilidad sin modificar lo existente |



**ABSTRACCIÓN**
REPOSITORIO-CLIENTES ES UNA INTERFAZ

**HERENCIA/COMPOSICIÓN**
IMPLEMENTACIONES QUE **REUTILIZAN** UNA CLASE BASE DE ACCESO A DATOS






**ENCAPSULAMIENTO**
LA CAPA DE SERVICIO NO CONOCE EL SQL




**POLIMORFISMO**
CAMBIAR REPOSITORIOSQL POR REPOSITORIOMEMORIA SIN TOCAR EL SERVICIO




**VÍNCULO CON SOLID**

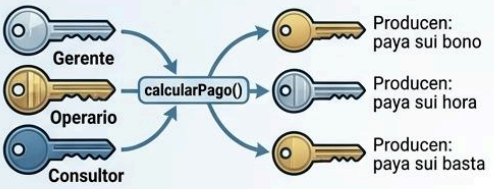



**VÍNCULO CON SOLID: S** (responsabilidad única) → encapsulamiento • **O** (abierto/cerrado) → polimorfismo • **L** (Liskov) → herencia correcta • **I** (segregación de interfaces) → abstracción • **D** (inversión de dependencias) → abstracción + polimorfismo

# GUÍA COMPLETA SOBRE EL POLIMORFISMO

## 5. DEFINICIÓN: Muchas Formas

Capacidad de que objetos de distintas clases respondan al **mismo mensaje** de forma propia.




## » TIPOS DE POLIMORFISMO

| Tipo | Momento | Mecanismo |
|---|---|---|
| **Sobrecarga** (*overloading*) | Compilación | Mismo nombre de método, distintos parámetros<br>• `calcularArea(double r) {}`<br>• `calcularArea(double w, double h) {}` |
| **Sobrescritura** (*overriding*) | Ejecución | La subclase **redefine** un método heredado<br>Superclass `calcularArea(){ return (; }` masked `calcularArea(double w) cadurn (; }` |

## » EJEMPLO PRÁCTICO EN CÓDIGO (JAVA)

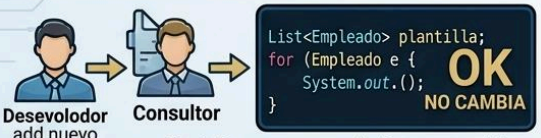
Conceptual de as clases abstractos, objetos objetos subclases reales

```
List<Empleado> plantilla = List.of(new Gerente(), new Operario());
for (Empleado e : plantilla) {
    System.out.println(e.calcularPago()); // cada objeto ejecuta SU versión
}
```




CADA OBJETO EJECUTA SU VERSIÓN

## BENEFICIOS ARQUITECTÓNICOS: EXTENSIBILIDAD




El código que recorre la lista **no cambia**

## PATRONES QUE LO APROVECHAN



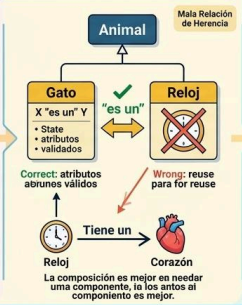

# 7. ERRORES FRECUENTES EN LA PROGRAMACIÓN ORIENTADA A OBJETOS

Guía para evitar malas prácticas de diseño

### 1. Atributos public o getters/setters automáticos para todo.



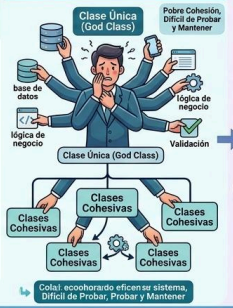

### 2. Herencia solo para reutilizar código, sin relación "es un".




### 3. Jerarquías de más de 3 niveles.




### 4. Clases con demasiadas responsabilidades (*God class*).




### 5. Uso de instanceof en lugar de polimorfismo.

